

The Refund That iPayMoney Does Not Document

The official documentation is now in the repository. It defines two payment endpoints, a webhook, and a sandbox. It defines **no customer refund of any kind** — and the word that looks like one means something else entirely.

Code state **3fe233d**CI **green**Unit tests **429**Browser tests **31**Refund questions answered **15 / 15**Documented refund operations **0**

THE SINGLE MOST IMPORTANT FINDING

The documentation uses « **remboursement** » — the ordinary French word for a refund — for an operation that is **not a refund**:

« Vous pouvez à tout moment demander depuis votre tableau de bord le **remboursement de vos avoirs**. Cela s'appelle un **versement**. » — extract L97

Your holdings, to **your** account. That is the merchant withdrawing their own balance to their own mobile-money or bank account, with a 50 000 FCFA floor and a three-day holding rule. Every one of the **30** occurrences of *remboursement* and *versement* in the document carries that sense. There is no other.

Reading it as a customer refund would have produced an integration that pays the merchant when a buyer is owed money. That is the trap this gate was called to check, and it is a real one.

A

CI status — the packet's caveat is now resolved

The implementation packet said CI had not run these commits and refused to claim a green badge it did not have. **It has now run, and it is green.**

RUN	COMMIT	JOB	RESULT
#31 · 32452577045	3fe233d	lint · typecheck · test	SUCCESS
#31	3fe233d	dependency scan	SUCCESS
#31	3fe233d	browser · authorization assertions	SUCCESS
#33 · 32456055226	4f47887	all three (docs-only commit)	SUCCESS

Every step inside run #31 concluded `success` , including the ones that matter most for this phase:

STEP	WHY IT MATTERS HERE
Rebuild the schema from scratch	Migration 0012 — two tables, three triggers, twelve CHECKs — applies to an empty database, not just to mine
Seed reference data	The three refund permissions and four clamped settings seed cleanly
pnpm test	429 tests on GitHub's PostgreSQL 16, not only on this container
Assert the ledger reconciles after the suite	Both detector probes proved they can still report a fault
Authorization assertions in a real browser	31 browser tests against a production build

WHAT THIS CHANGES IN THE EARLIER PACKET

Section 13 and section 16 of the implementation packet both say CI has not run. **That statement was true when written and is now superseded by this report.** The packet is not being quietly edited: it is the record of what was known at the time, and this is the record of what is known now.

Phase 3 code status — unchanged, and re-verified

No implementation code was modified during this gate. The tree at `3fe233d` is exactly what CI ran and what the implementation packet describes. Re-run locally after the documentation arrived:

COMMAND	OUTPUT	
<code>pnpm lint</code>	5 packages · Done, 0 warnings	CLEAN
<code>pnpm typecheck</code>	5 packages · Done	CLEAN
<code>pnpm test</code>	23 files · 429 passed 3 skipped (432)	GREEN
<code>tsx scripts/reconcile-check.ts</code>	both detector probes behaved; ledger clean	GREEN

The 3 skipped tests are the iPayMoney sandbox integration tests. They remain skipped for the reason they have always been skipped — **no sandbox credentials exist in this repository** — and the arrival of documentation does not change that. **No sandbox call has been made.**

The provider abstraction, checked against the confirmed payment API

Requested mapping, and how the current contract answers it:

AFRINEXT	IPAYMONEY	FITS THE CURRENT CONTRACT?
<code>ChargeInput.idempotencyKey</code> <code>= order:{id}:charge</code>	<code>transaction_id</code>	YES — derived, never caller-supplied. iPayMoney rejects a reuse with 422, which is safe: it cannot double-charge
<code>payments.provider_ref</code>	<code>reference</code>	YES — already UNIQUE (provider, provider_ref), and already the key an inbound event is matched on
<code>ChargeStatusValue</code>	<code>status</code>	PARTLY — only <code>succeeded</code> and <code>failed</code> appear in examples; the full enumeration is undocumented (F5)
<code>webhook → order</code>	<code>external_reference</code>	YES — <code>external_reference</code> is our <code>transaction_id</code> echoed back, so it resolves to the order; the event is matched on <code>reference</code> and this corroborates it
<code>ChargeStatus.amount</code> (required)	— absent —	NO — see below
<code>VerifiedEvent.providerEventId</code>	— absent —	NO — see below

TWO CONTRACT MISMATCHES FOUND – REPORTED, NOT FIXED

1. `ChargeStatus.amount` is required and `iPayMoney` never sends an amount. Neither the webhook payload nor the payment-status response carries one. A faithful `getCharge()` cannot be written: the adapter would have to fill the amount in from our own database, which is us telling ourselves what we already believe rather than the provider corroborating it.

2. `VerifiedEvent.providerEventId` has no counterpart. Our replay defence is `UNIQUE (provider, provider_event_id)`. `iPayMoney` sends no event id and retries five times on non-2xx, so duplicate deliveries are certain rather than hypothetical. The adapter would have to derive an id from the fields the event does carry.

Neither is a defect in the Phase 3 refund implementation, and neither has been changed. They are properties of the charge contract that the new documentation exposes, and they belong to the `iPayMoney` integration milestone. Deriving an event id is sound — a repeat of the same `(reference, status)` fact is exactly what deduplication should collapse — but it is a decision to take deliberately, not silently.

ONE THING THIS GATE DELIBERATELY DID NOT TIDY

The header comment in `packages/core/src/payments/ipaymoney.ts` still says the documentation "is not available in this repository". **That sentence is now false.** It is a comment, not behaviour, and correcting it means touching an implementation file during a gate that was told not to. It is listed here so it is a visible decision rather than an oversight, and it is the first item of the next milestone.

Two endpoints exist in the entire document. There are no others, and no other HTTP method appears anywhere.

Create a payment · POST <https://i-pay.money/api/v1/payments>

DOCUMENTED

HEADER	VALUE
Ipay-Payment-Type	mobile · card
Ipay-Target-Environment	sandbox · live
Authorization	Bearer <secret key>
Content-Type	application/json

BODY FIELD	TYPE	DOCUMENTED AS
customer_name	string	the customer's name
currency	string	XOF
country	string	country code — NE for Niger, BJ for Benin
amount	string	"must be greater than 100"
transaction_id	string	a reference for the transaction we want performed
msisdn	string	the payer's number

```
200 — success (L179–L185)
{ "status": "succeeded", "reference": "vslfxgawkpkm" }
```

STATUS	DOCUMENTED MESSAGE
400	Incorrect MSISDN · Missing params · Not Allowed Environment · Not Allowed Payment Type · Missing Authentication Key
401	Unauthorized: No Valid Key
403	Forbidden: Environment Not Available
406	Not Acceptable: Invalid Content Type
422	External Reference Not Valid — reference invalid, or already exists

No 5xx is documented anywhere, and no meaning is given for one. That matters directly: our classifier only produces `failed` from a documented provider statement that nothing was executed, and no such statement exists for a server error.

Query a payment · GET https://i-pay.money/api/v1/payments/{reference} DOCUMENTED

The path parameter is *"la référence que vous avez reçu en réponse lors du Post"* — iPayMoney's reference, not ours.

```
200 (L241-L249)
{
  "external_reference": "Reference-13409",
  "reference": "vslfxgawkpkm",
  "status": "failed",
  "msisdn": "22787505050"
}
```

A DESIGN DECISION THIS DOCUMENTATION VALIDATES

The POST returns "status": "succeeded" **synchronously**, and iPayMoney *also* sends a webhook for the same fact. Phase 2 deliberately clamps a provider's synchronous answer to pending and confirms nothing until a verified event arrives. That was written against a mock, on the argument that a synchronous answer travels on a connection the caller may control. The real provider turns out to behave exactly the way that argument assumed — the boundary was built for the right shape.

Fees DOCUMENTED

MOBILE-MONEY COLLECTION	3 % of the amount, taken at collection
VISA / MASTERCARD	3 % of the purchase
REVERSEMENT TO MOBILE MONEY, INSIDE NIGER	no fee
REVERSEMENT TO A BANK ACCOUNT	7 000 FCFA flat, whatever the amount
REVERSEMENT THRESHOLD	≥ 50 000 FCFA, and the funds must have sat on the merchant account more than 3 days

The 3 % is charged on the way in. It is not stated anywhere whether it is returned if a payment is refunded — which is question K11, and it decides whether a full refund costs Afrinext 3 % of the order.

Webhook facts — including one that is a security finding

FACT	DOCUMENTED
Configured per-URL in the dashboard, under Développeurs → Webhooks	YES
Exactly two event types exist: success and failure	YES — stated positively, L321
Our endpoint must answer <code>2xx</code> ; anything else, 3xx included, counts as not received	YES
Retries 5 times on non-receipt, 500 ms apart and increasing	YES
Payload fields: <code>external_reference</code> , <code>reference</code> , <code>status</code> , <code>msisdn</code>	YES
Amount in the payload	ABSENT
Event id in the payload	ABSENT
Timestamp / tolerance window	ABSENT

```
success payload (L359–L368)
{ "data": { "external_reference": "random-38",
            "reference": "yeweyk6rd7cm",
            "status": "succeeded",
            "msisdn": "40410000001" } }
```

The signature, and why it is a finding

The prose and the example **disagree**, and both readings are worse than what our boundary is built to verify.

SOURCE	SAYS
Prose (L332)	"Il est introduit dans l'en-tête de chaque requête <code>x-iPayMoney-secret</code> une signature"
Example (L338)	<code>"secret-hash": "sk_58951rhguiq859905dfn5903gh"</code>
Setup instructions	"Remplissez votre <i>Secret Hash</i> . C'est votre clé API secrète " — copy it from the API Keys page

WHAT THIS APPEARS TO MEAN, AND WHY IT MUST BE ASKED

The header name differs between prose and example. More seriously, the example's value is the **secret key itself** (`sk_...`), and the setup text says the Secret Hash is the API secret. If that is what it means, then this is a **static shared secret echoed in a header — not a signature over the request body**.

Two consequences follow, and neither is theoretical:

- **The check does not cover the payload.** Anyone holding the secret can forge any event body. A signature over the raw bytes cannot be forged without also producing a matching digest; a constant header can.
- **The secret is transmitted on every single webhook**, to our endpoint, five times over on retry — so it appears in any proxy log, any error report, any request dump.

Afrinext's webhook boundary verifies a signature over the raw bytes before parsing, and that property is load-bearing: it is why a forged amount cannot confirm a payment. **Combined with finding F1 — no amount anywhere — a forged event would face no amount check either.** This is question K1, and it should be asked before any webhook is trusted with money.

E Refund facts — fifteen questions, answered from the document only

Every answer below was read out of the supplied documentation. **None is inferred**, and none is filled in from another provider's conventions.

#	QUESTION	ANSWER	BASIS
R1	Endpoint to refund a customer payment	NOT DOCUMENTED	Two endpoints exist in the whole document; both are <code>/api/v1/payments</code>
R2	HTTP method for creating a refund	NOT DOCUMENTED	Only <code>POST</code> and <code>GET</code> appear anywhere, both on payments
R3	Required refund request body	NOT DOCUMENTED	—
R4	Which identifier a refund uses	NOT DOCUMENTED	No refund operation exists to take one
R5	Refund response and refund reference	NOT DOCUMENTED	—
R6	Refund status endpoint	NOT DOCUMENTED	The documented <code>GET</code> is a payment status endpoint and is not one
R7	Refund webhook	NOT DOCUMENTED	Stronger than silence: the document states that only two event types exist , success and failure on a transaction
R8	Refund idempotency mechanism	NOT DOCUMENTED	—
R9	Querying a refund by refund ref · payment ref · external_reference · our key	NOT DOCUMENTED	—
R10	Behaviour on refund success · rejection · timeout · network failure · 5xx · duplicate	NOT DOCUMENTED	For <i>payments</i> only 400/401/403/406/422 are documented. No timeout, connection-failure, 5xx or duplicate semantics are documented anywhere in the document , for any operation
R11	Sandbox refund support	NOT DOCUMENTED	The sandbox table lists payment scenarios only
R12	Mobile Money refund support	NOT DOCUMENTED	Mobile money is documented as a <i>collection</i> channel
R13	Visa / Mastercard refund support	NOT DOCUMENTED	Cards are documented as a <i>collection</i> channel
R14	Refund processing time	NOT DOCUMENTED	—
R15	Refund fees	NOT DOCUMENTED	The fees section covers purchase fees and reversement fees only

The one adjacent thing in the document, which is not a refund

« le marchand peut être amené à mettre fin à **une opération en cours**. Cela peut se faire à travers le menu Paiements. » — L304

A merchant may end an operation **in progress**, from the **dashboard menu**. It has no API, no method, no request or response schema, no identifier, and no stated financial effect. What it actually does is **UNCLEAR**, and it is **not** a refund of a completed payment. It is recorded here so nobody later mistakes it for one.

CONCLUSION OF THE REFUND INVESTIGATION

The supplied documentation contains no customer-refund operation of any kind. Not an endpoint, not a method, not a payload, not a status, not a webhook, not a sandbox scenario, not a fee, not a processing time.

TWO THINGS THIS REPORT REFUSES TO DO

The documented `GET /api/v1/payments/{reference}` is **not** presented anywhere in this report as a refund-status endpoint, and the documented **reversement** is **not** presented anywhere as customer-refund functionality. Both would have been easy and both would have been false.

Unknowns — what must be confirmed by iPayMoney support

These are gaps in the *payment* documentation, found by checking it against our provider contract rather than by reading it on its own.

#	UNKNOWN	CONSEQUENCE
F1	No amount in the webhook, and none in the payment-status response	Our webhook boundary cross-checks the amount exactly when the event carries one. iPayMoney carries none in either channel, so the check can never run and the charged amount is not independently verifiable from the provider at all
F2	No event id in the webhook	Our replay defence is <code>UNIQUE (provider, provider_event_id)</code> . With no id and five guaranteed retries, duplicates are certain; the adapter must derive an id
F3	No lookup by our own identifier	<code>GET</code> takes iPayMoney's <code>reference</code> . If the <code>POST</code> response is lost we hold no reference and there is no documented way to ask "what became of my <code>transaction_id</code> ?" — the payment equivalent of the refund problem below
F4	<code>transaction_id</code> vs <code>external_reference</code>	We send one name and receive another. The dashboard section additionally says the external reference is "générée de façon automatique", which contradicts us supplying it
F5	The complete status enumeration	Only <code>succeeded</code> and <code>failed</code> appear in examples. The sandbox names five <i>scenarios</i> , which are not status values. Reconciliation cannot be written against a partial enum
F6	Webhook header name and signature scheme	Prose says <code>x-iPayMoney-secret</code> ; the example shows <code>secret-hash</code> carrying the secret itself. See §D — this is the security finding
F7	Amount units	<code>amount</code> is a string that must exceed 100. XOF has zero decimals so whole francs is the only sensible reading, but the document does not say it. A wrong assumption here is a 100× error
F8	No 5xx semantics	Nothing says what a 5xx means for the transaction — which is exactly the case our classifier is built to treat as unknown, and which the documentation therefore cannot resolve
F9	Rate limits	Not documented for either endpoint
F10	Settlement timing to the merchant account	Reversement frequency is configurable, but the T+N at which a collected payment becomes part of the available balance is not stated. This is a Settlement input

Sandbox — what could be tested in a future milestone

STATED BEFORE ANYTHING ELSE

No sandbox call has been made from this repository, ever. Everything below is what the documentation says is available, not what has been observed. The three skipped integration tests remain skipped because no credentials exist here.

The documented test MSISDNs, verbatim:

MSISDN	SCENARIO	WHAT IT WOULD EXERCISE IN OUR BOUNDARY
40410000000 · 40410000001	success	The whole confirmation path: charge → webhook → paid → entitlement → frozen fee schedule
40410000002 · 40410000003	error	A provider-side error, and how it is classified
40410000004 · 40410000005	insufficient_fund	A definite business rejection — the shape that <i>could</i> carry not-executed evidence
40410000006 · 40410000007	declined	A second definite rejection, distinct from the first
40410000008 · 40410000009	pending (180 s)	The most valuable one. Three minutes of genuine in-flight state — the late-payment grace window, the expiry race, and the webhook arriving after our timer, against a real provider rather than a mock

What a sandbox milestone could establish, and what it could not

COULD BE ESTABLISHED BY SANDBOX TESTING	COULD NOT BE
- The real status enumeration (F5) — by observing all five scenarios - The actual webhook header name and whether the value varies with the body (F6) - Whether the amount is echoed anywhere after all (F1) - Whether a duplicate <code>transaction_id</code> really 422s (F3, F4) - Whether whole francs is right (F7) — by charging 100 and reading the dashboard - Retry behaviour: return 500 deliberately and count the deliveries	Anything about refunds. There is no refund operation to call, so no sandbox scenario can exercise one 5xx semantics (F8) — a sandbox will not produce a server error on demand - Settlement timing (F10) — sandbox money does not settle - Live-mode behaviour of any kind

Live activation — the documented requirements

Recorded because they are a lead time, not a task. **Nothing has been implemented or prepared against them.** Until the submission is approved, the account is **sandbox only**.

Enable Live in the dashboard, then submit under **Paramètres → Information Légale**:

INFORMATION TO ENTER	DOCUMENTS TO UPLOAD
• Nom de l'entreprise • Numéro de téléphone • Adresse du siège social • Secteur d'activité • Site web / Application • Numéro : ID card / passeport • N° du registre de commerce • Numéro IFU/NIF (numéro fiscal)	• RCCM • IFU/NIF (numéro fiscal) • Pièce d'identité

Save, then submit. The answer arrives **by email after examination** — the document gives no service-level for that review, so the wait is an unknown that should be started early rather than discovered late.

TWO OF THESE ARE WORTH NOTICING NOW

"**Site web / Application**" is required, which means iPayMoney will look at Afrinext before approving. And **AFRI NEXT TECHNOLOGIE is an Entreprise Individuelle** — whether that changes which documents are wanted, and whose *pièce d'identité*, is worth confirming before the submission rather than after a rejection.

I What this means operationally — and whether the design still holds

You asked specifically whether the existing Phase 3 design **safely handles unknown refund capabilities**. It does, and the reason is structural rather than lucky.

DESIGN DECISION	MADE WHEN REFUNDS WERE AN UNKNOWN	AGAINST THE REAL DOCUMENTATION
<code>getRefund()</code> is optional on the interface, and callers branch on <code>supportsRefundQuery()</code>	To avoid asserting a capability we could not confirm	CORRECT — there is no refund query, so the false branch is the real one
An <code>in_doubt</code> refund is never retried , only resolved	Because a timeout is not evidence	CORRECT — and now load-bearing, because no mechanical resolution exists at all
Manual reconciliation exists as a first-class path, with dual control and an external reference	As the fallback for a provider that cannot be asked	CORRECT — it is not the fallback. It is the only path.
The refund's <code>provider</code> , amount and currency are frozen by a trigger	To keep a snapshot honest	CORRECT — and more valuable now, since the provider corroborates no amount anywhere
Refunds post nothing to the ledger	Because Phase 2 posts no capture	CORRECT — and now also avoids booking a reversal for an operation that has no provider mechanism
<code>refund()</code> and <code>getRefund()</code> on the <code>iPayMoney</code> adapter throw	Because nothing was known	CORRECT — and must keep throwing, because nothing is known <i>still</i>

THE OPERATIONAL CONSEQUENCE, STATED PLAINLY

The Phase 3 design named a scenario as its worst case: *"if both R3 and R4 are no, every timed-out refund becomes a human reading a statement."*

The documentation is worse than that scenario. It is not that ambiguous refunds need a person — it is that **every refund needs a person, from the very first one**, because there is no API to attempt one through. Afrinext currently has **no automated way to return money to a buyer** through `iPayMoney`.

The machinery built in Phase 3 is not wasted by this — the debt is still recorded automatically and inside the confirming transaction, the queue is still age-sorted, the evidence requirements and dual control still apply, and reconciliation still reports a debt nobody paid. What changes is which path carries the volume: **manual reconciliation is not the exception, it is the process.**

What that costs, concretely

QUESTION	ANSWER TODAY
How does a buyer's money actually get back to them?	Unknown. Not through any documented API. Possibly a merchant-initiated mobile-money transfer outside iPayMoney, possibly an iPayMoney operation that exists but is undocumented — question K5
How many people does it take?	Two, wherever two hold <code>refund.reconcile_manual</code> . Afrinext has one finance user today, so the system records <code>single_operator</code> and reconciliation flags it every run
How does it scale?	It does not. It is linear in refunds, and refunds are linear in late payments — which are linear in how unreliable mobile-money confirmation is in Niger
Is the 3 % collection fee returned on a refund?	Not documented — question K11. If not, a full refund costs Afrinext 3 % of the order out of its own funds

SETTLEMENT IS NOT STARTED, AND THIS REPORT DOES NOT START IT

Nothing in this gate touched `ledger/flows.ts` , `ledger/post.ts` , settlement holds or payout logic. What follows is a list of what would block Settlement if it were authorised, so that the list exists before the work does.

#	BLOCKER	WHY SETTLEMENT CANNOT PROCEED PAST IT
S1	Phase 3 has not been approved	The standing rule: Settlement waits on this gate
S2	Settlement timing (T+N) is undocumented – F10	Settlement decides when a collected payment becomes a seller's claim. Posting that before Afrinext actually holds the funds is a promise the balance cannot keep, which is the exact thing Phase 2 refused to do
S3	The amount is not verifiable from the provider – F1	The ledger's first commandment is that the books are the truth. Booking a capture against an amount no provider will confirm makes the ledger authoritative about a number only we assert
S4	The 3 % fee's treatment on refund is unknown – K11	Settlement must post the provider's fee somewhere. If a refund does not return it, the reversal is not the mirror of the capture and the accounts must say so
S5	There is no refund execution to reverse	Settlement was to post a capture and its reversal. The reversal half has no provider mechanism, so a refunded sale's ledger treatment depends on how money actually leaves – which is K5
S6	No real payment has ever been made	Every capture in every test came from the mock. Settlement posting real money against a path never exercised against the real provider is the largest untested step the platform would have taken

S2, S3, S4 and S5 are all answered by iPayMoney, not by us. That is the case for asking before building.

The exact questions to send iPayMoney

Ordered by what they block. The first five decide whether Afrinext can refund a buyer at all; the rest decide whether the payment integration is safe and what Settlement can assume.

Refunds — these block a buyer getting their money back

#	QUESTION	BLOCKS
K1	Is there any API to refund a customer payment, in whole or in part? If yes: endpoint, method, request body, response, and which identifier it takes. If no, please say so explicitly.	Everything below
K2	If a refund API exists, can its status be queried afterwards, and by which identifier — the refund's reference, the original payment's <code>reference</code> , our <code>transaction_id</code> , or an idempotency key we supply?	Whether an ambiguous refund can ever be resolved mechanically
K3	If a refund API exists, does it accept an idempotency key from us, and does a repeat of the same key return the original outcome rather than creating a second refund?	Whether a lost response can be retried safely
K4	If a refund API exists, which error codes mean, in your words, that no refund was created? We treat any failure we cannot prove as unknown, and never retry an unknown.	Whether a rejection is retryable or becomes manual
K5	If there is no refund API: what is the correct way for a merchant to return money to a buyer? A dashboard operation, a support request, or a transfer outside iPayMoney? And is it visible in the API or reports afterwards, so we can reconcile it?	Whether Afrinext can refund at all
K6	Are refunds possible on both mobile money and Visa/Mastercard, and is there a time limit after the original payment?	Whether an old <code>refund_due</code> is refundable

Security and correctness of the payment integration

#	QUESTION	BLOCKS
K7	Exactly how is a webhook authenticated? The documentation names the header <code>x-iPayMoney-secret</code> in prose but shows <code>secret-hash</code> in the example, carrying what appears to be the secret key itself. Is it (a) the static secret echoed in a header, or (b) an HMAC computed over the request body? If (a), is a body signature available? A constant header does not protect the payload.	Whether a webhook can be trusted with money
K8	Is the amount available anywhere — in the webhook payload or the payment-status response? Neither carries one today, so we cannot verify that the amount charged is the amount we requested.	The amount cross-check
K9	Do webhooks carry a unique event id? You retry five times, so we will receive duplicates; without an id we must derive one to deduplicate safely.	Replay protection
K10	Can a payment be looked up by our <code>transaction_id</code> rather than your <code>reference</code>? If a POST response is lost we hold no reference and cannot ask what happened.	Recovering a lost charge
K11	Is the 3 % collection fee returned when a payment is refunded or reversed?	Whether a refund costs Afrinext 3 %
K12	What is the complete list of <code>status</code> values a payment can take? We have seen <code>succeeded</code> and <code>failed</code> ; the sandbox names scenarios rather than statuses.	Reconciliation against a full enum
K13	Is <code>amount</code> in whole francs? XOF has no decimal places; we want this confirmed rather than assumed, because the wrong reading is a 100× error.	Every amount we ever send
K14	What does a 5xx from your API mean for the transaction — that it was not created, or that its outcome is undetermined? And are there rate limits on either endpoint?	How we classify a server error
K15	After how long (T+N) does a collected payment become part of the available balance eligible for reversal?	Settlement
K16	For an <i>Entreprise Individuelle</i> , which Live-mode documents apply, and whose <i>pièce d'identité</i> ?	Live activation

K1 AND K5 ARE THE TWO THAT MATTER MOST

Everything else is an integration detail that changes how carefully we write an adapter. **K1 and K5 decide whether a buyer whose payment arrived too late can be given their money back by any means at all** — and Afrinext already has the machinery to owe that money, record it, and refuse to lose track of it. What it does not have is a way to send it.

Recommended next milestone

NOT SETTLEMENT

Settlement is blocked by six things (\$J), four of which only iPayMoney can answer. Starting it now would mean writing ledger postings against assumptions about settlement timing, fee treatment and amount verification that the documentation does not support.

Recommended: send the questions, then integrate payments against the sandbox

In that order, and the first step costs nothing but an email.

1 Send K1–K16 to `support@i-pay.money`

K1 and K5 first. Until they are answered, no amount of code changes the fact that there is no documented way to refund a buyer. This is not a development task and it should not wait for one.

2 iPayMoney payment integration, sandbox only

Implement `createCharge`, `getCharge` and `verifyWebhook` against the two documented endpoints, and drive all five sandbox scenarios — especially the 180-second `pending`, which exercises the late-payment grace window against a real provider instead of a mock. `refund()` and `getRefund()` keep throwing, because nothing has changed about them.

Two contract questions must be decided deliberately in that milestone, not silently: how a `providerEventId` is derived when the provider sends none, and what `getCharge()` does about an amount the provider will not state. Both are in \$B.

3 Start the Live activation submission in parallel

It is a queue with an unknown wait, not a task. Beginning it during the sandbox milestone costs nothing and may save weeks.

4 Then Settlement

With T+N confirmed, fee treatment confirmed, and at least one real payment observed end to end.

Two smaller corrections owed

ITEM	WHY IT WAS NOT DONE IN THIS GATE
The header comment in <code>payments/ipaymoney.ts</code> still says the documentation is unavailable. It is now false.	Correcting it means editing an implementation file during a gate instructed not to. First item of the next milestone
The step-up re-verification screen still does not exist	Elevation is enforced and its refusal is proven in a browser, but a finance user can only obtain elevation through the database. Unchanged since the implementation packet

GATE SUMMARY

CI green at 3fe233d. Phase 3 code unchanged and re-verified. The provider abstraction fits the documented payment API in four of six mappings, with two mismatches reported and not fixed. **iPayMoney documents no customer refund of any kind**, and the Phase 3 design handles that correctly because it never assumed otherwise.

Nothing merged. Settlement not started. No iPayMoney code written. No sandbox call made.

Phase 3 · final gate report · branch `claude/create-app-repository-1cxsih` · code state `3fe233d` · documentation `docs/providers/ipaymoney/`